Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Мухамедьянова А.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.10.26

Москва, 2026

Постановка задачи

Вариант 16.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Вариант №16: Задается радиус окружности. Необходимо с помощью метода Монте-Карло рассчитать ее площадь.

Общий метод и алгоритм решения

Использованные функции и библиотечные средства:

- int strcmp(const char *str1, const char *str2); — сравнивает строки для определения названия переданного аргумента.
- int sscanf(const char *str, const char *format, ...); — преобразует строковые аргументы командной строки в числовые значения.
- void *malloc(size_t size); — выделяет динамическую память для массивов потоков и их аргументов.
- void free(void *ptr); — освобождает ранее выделенную динамическую память.
- int pthread_create(...); — создает новый поток выполнения, которому передается функция work.
- int pthread_join(pthread_t thread, void **retval); — ожидает завершения указанного потока.
- void *work(void *ptr); — функция вычислений, выполняющая метод Монте-Карло для оценки площади круга.
- unsigned int rand_r(unsigned int *seed); — генерирует псевдослучайные числа с использованием отдельного начального значения для каждого потока.
- int clock_gettime(clockid_t clock_id, struct timespec *tp); — получает время для измерения длительности последовательных и параллельных вычислений.

Программа вычисляет приближенное значение площади круга двумя способами: последовательно в одном потоке и параллельно с использованием заданного количества потоков. Для вычислений используется метод Монте-Карло.

1) Инициализация программы и обработка аргументов командной строки

- Инициализируются переменные radius, n_points и n_threads, отвечающие за радиус круга, количество случайных точек и количество потоков.
- Проверяется корректность количества переданных аргументов командной строки.
- С помощью strcmp() определяется тип каждого аргумента: --radius, --points или --threads.
- С помощью sscanf() строковые значения преобразуются в числовые параметры.
- Выполняется проверка корректности параметров.
- При обнаружении некорректных аргументов программа выводит сообщение об ошибке и завершает работу.

2) Последовательное вычисление площади круга

- Создается структура ThreadArgs, содержащая количество точек, число попаданий внутрь круга, радиус и начальное значение генератора псевдослучайных чисел.
- С помощью clock_gettime() фиксируется время начала вычислений.

- Вызывается функция work(&serial), которая выполняет вычисления последовательно в основном потоке.
- Для каждой точки генерируются координаты \(x\) и \(y\) в пределах квадрата со стороной \(2r\).
- Проверяется условие попадания точки внутрь круга: \(x^2+y^2\leq r^2\).
- Если условие выполняется, счетчик попаданий увеличивается.
- После обработки всех точек фиксируется время окончания вычислений и рассчитывается продолжительность последовательной работы.

3) Параллельное вычисление площади круга

- С помощью malloc() выделяется память для массива идентификаторов потоков threads и массива структур с параметрами args.
- Проверяется успешность выделения памяти. При ошибке программа освобождает выделенные ресурсы и завершает работу.
- Количество точек распределяется между потоками примерно поровну. Если количество точек не делится на количество потоков без остатка, оставшиеся точки распределяются по одной между первыми потоками.
- Для каждого потока задаются количество обрабатываемых точек, радиус, начальное значение генератора случайных чисел и начальное значение счетчика попаданий.
- С помощью clock_gettime() фиксируется время начала параллельных вычислений.
- В цикле вызывается pthread_create(), создающий отдельные потоки для выполнения функции work.
- Каждый созданный поток независимо генерирует случайные точки и подсчитывает количество попаданий в круг.
- Последний набор точек обрабатывается непосредственно основным потоком посредством вызова work(&args[n_threads - 1]). Таким образом, основной поток также участвует в вычислениях, а создавать дополнительный поток для последней части работы не требуется.
- После завершения вычислений основного потока вызывается pthread_join() для каждого созданного потока, чтобы дождаться завершения их работы и получить окончательные результаты.
- Фиксируется время окончания параллельных вычислений и рассчитывается продолжительность параллельной работы.

Если создать один из потоков не удается, программа выводит сообщение об ошибке, дожидается завершения уже созданных потоков, освобождает выделенную память и завершает работу.

5) Объединение результатов и вычисление площади

- С помощью цикла суммируются значения hits всех потоков, формируя общее количество попаданий точек внутрь круга.
- Вычисляется площадь квадрата, в котором генерировались точки:

$$S_{\text{квадрата}} = 4r^2$$

- Приближенная площадь круга определяется как произведение площади квадрата на отношение количества попаданий к общему количеству точек:

$$S_{\text{круга}} = S_{\text{квадрата}} * \frac{N_{\text{попаданий}}}{N_{\text{всего}}}$$

- Площадь вычисляется отдельно для последовательного и параллельного вариантов программы.

6) Оценка эффективности параллельных вычислений

- Вычисляется ускорение параллельного алгоритма относительно последовательного:

$$S = \frac{T_{\text{последовательное}}}{T_{\text{параллельное}}}$$

$$E = \frac{S}{p}$$

- Вычисляется эффективность использования потоков:

- С помощью printf() выводятся заданный радиус, количество точек, полученные значения площади круга, время последовательных и параллельных вычислений, ускорение и эффективность параллельной обработки.

Код программы

Lab2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    size_t n_points;
    size_t hits;
    double radius;
    unsigned int seed;
} ThreadArgs;

void *work(void *ptr)
{
    ThreadArgs *args = (ThreadArgs *)ptr;
    unsigned int seed = args->seed;
    size_t hits = 0;
    double r = args->radius;

    for (size_t i = 0; i < args->n_points; ++i) {
        double x = (2.0 * rand_r(&seed) / (RAND_MAX + 1.0) - 1.0) * r;
        double y = (2.0 * rand_r(&seed) / (RAND_MAX + 1.0) - 1.0) * r;

        if (x * x + y * y <= r * r)
            ++hits;
    }
```

```
    args->hits = hits;
    return NULL;
}

int main(int argc, char **argv)
{
    double radius = -1.0;
    size_t n_points = 0;
    size_t n_threads = 0;

    if ((argc - 1) % 2 != 0) {
        printf("Example: ./lab2 --radius 5 --points 10000000 --threads 4\n");
        return 1;
    }

    for (int i = 1; i < argc; i += 2) {
        if (strcmp(argv[i], "--radius") == 0) {
            if (sscanf(argv[i + 1], "%lf", &radius) != 1) return 1;
        } else if (strcmp(argv[i], "--points") == 0) {
            if (sscanf(argv[i + 1], "%zu", &n_points) != 1) return 1;
        } else if (strcmp(argv[i], "--threads") == 0) {
            if (sscanf(argv[i + 1], "%zu", &n_threads) != 1) return 1;
        } else {
            printf("Unknown argument: %s\n", argv[i]);
            return 1;
        }
    }

    if (radius <= 0 || n_points == 0 || n_threads == 0 || n_threads > n_points) {
```

```
    printf("Need radius > 0, points >= 1, 1 <= threads <= points\n");
    return 1;
}

struct timespec start, end;
ThreadArgs serial = {n_points, 0, radius, 12345};

clock_gettime(CLOCK_MONOTONIC, &start);
work(&serial);
clock_gettime(CLOCK_MONOTONIC, &end);
double serial_ms = (end.tv_sec - start.tv_sec) * 1000.0
                 + (end.tv_nsec - start.tv_nsec) / 1000000.0;

pthread_t *threads = malloc((n_threads - 1) * sizeof(pthread_t));
ThreadArgs *args = malloc(n_threads * sizeof(ThreadArgs));
if ((n_threads > 1 && threads == NULL) || args == NULL) {
    printf("Memory allocation failed\n");
    free(threads);
    free(args);
    return 1;
}

for (size_t i = 0; i < n_threads; ++i) {
    args[i] = (ThreadArgs){
        .n_points = n_points / n_threads + (i < n_points % n_threads),
        .hits = 0,
        .radius = radius,
        .seed = 12345u + (unsigned int)i * 7919u
    };
}
```

```
clock_gettime(CLOCK_MONOTONIC, &start);

size_t created = 0;
for (size_t i = 0; i + 1 < n_threads; ++i) {
    if (pthread_create(&threads[i], NULL, work, &args[i]) != 0) {
        printf("Could not create thread\n");
        for (size_t j = 0; j < created; ++j)
            pthread_join(threads[j], NULL);
        free(args);
        free(threads);
        return 1;
    }
    ++created;
}

work(&args[n_threads - 1]);

for (size_t i = 0; i < created; ++i)
    pthread_join(threads[i], NULL);

clock_gettime(CLOCK_MONOTONIC, &end);
double parallel_ms = (end.tv_sec - start.tv_sec) * 1000.0
                   + (end.tv_nsec - start.tv_nsec) / 1000000.0;

size_t total_hits = 0;
for (size_t i = 0; i < n_threads; ++i)
    total_hits += args[i].hits;

double square_area = 4.0 * radius * radius;
```

```
    double serial_area = square_area * (double)serial.hits / (double)n_points;
    double parallel_area = square_area * (double)total_hits / (double)n_points;
    double speedup = serial_ms / parallel_ms;
    double efficiency = speedup / n_threads;

    printf("Radius: %.2f; points: %zu\n", radius, n_points);
    printf("Sequential area: %.6f; time: %.3f ms\n", serial_area, serial_ms);
    printf("Parallel area:   %.6f; time: %.3f ms\n", parallel_area, parallel_ms);
    printf("\nThreads | Time (ms) | Speedup | Efficiency (%%)\n");
    printf("%7zu | %9.3f | %7.3f | %14.2f\n",
           n_threads, parallel_ms, speedup, efficiency * 100.0);

    free(args);
    free(threads);
    return 0;
}
```

Протокол работы программы

Тестирование:

```
allie@parsley:~$ cc lab2.c -o lab2
allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 4
Radius: 5.00; points: 100000000
Sequential area: 78.537961; time: 1160.738 ms
Parallel area:   78.538846; time: 358.284 ms

Threads | Time (ms) | Speedup | Efficiency (%)
      4 |   358.284 |   3.240 |          80.99
allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 10
Radius: 5.00; points: 100000000
Sequential area: 78.537961; time: 1151.372 ms
```

Parallel area: 78.541798; time: 267.409 ms

Threads | Time (ms) | Speedup | Efficiency (%)

10 | 267.409 | 4.306 | 43.06

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 15

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1173.351 ms

Parallel area: 78.541970; time: 265.493 ms

Threads | Time (ms) | Speedup | Efficiency (%)

15 | 265.493 | 4.420 | 29.46

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 18

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1168.782 ms

Parallel area: 78.541061; time: 265.673 ms

Threads | Time (ms) | Speedup | Efficiency (%)

18 | 265.673 | 4.399 | 24.44

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 20

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1209.877 ms

Parallel area: 78.541746; time: 272.962 ms

Threads | Time (ms) | Speedup | Efficiency (%)

20 | 272.962 | 4.432 | 22.16

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 128

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1228.656 ms

Parallel area: 78.540600; time: 250.014 ms

Threads | Time (ms) | Speedup | Efficiency (%)

128 | 250.014 | 4.914 | 3.84

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 256

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1180.118 ms

Parallel area: 78.537092; time: 267.150 ms

Threads | Time (ms) | Speedup | Efficiency (%)

  256 | 267.150 | 4.417 | 1.73

allie@parsley:~$ ./lab2 --radius 5 --points 100000000 --threads 1024

Radius: 5.00; points: 100000000

Sequential area: 78.537961; time: 1186.148 ms

Parallel area: 78.545729; time: 288.550 ms

Threads | Time (ms) | Speedup | Efficiency (%)

  1024 | 288.550 | 4.111 | 0.40

allie@parsley:~$

Таблица тестирования:

Количество потоков	Время выполнения (мс)	Ускорение	Эффективность(%)
4	358,284	3,240	80,99
10	267,409	4,306	43,06
15	265,493	4,420	29,46
18	265,673	4,399	24,44
20	272,962	4,432	22,16
128	250,014	4,914	3,84
256	267,150	4,417	1,73
1024	288,550	4,111	0,40

Вывод

В ходе лабораторной работы была успешно реализована программа на языке C для параллельных вычислений с использованием библиотеки pthread. Программа вычисляет приближённую площадь круга методом Монте-Карло как последовательно, так и с использованием нескольких потоков. Были реализованы распределение вычислительной нагрузки между потоками, измерение времени выполнения, а также расчёт ускорения и эффективности параллельных вычислений. В результате экспериментов установлено, что увеличение количества потоков позволяет сократить время выполнения до определённого предела, после которого дальнейшее увеличение их числа не приводит к существенному ускорению и может снижать производительность. Все поставленные в работе задачи были выполнены в полном объёме.